CHANDIGARH UNIVERSITY

Department of Computer Applications

Minor Project Report

"Automation Testing of CareerHub – Job Portal Using Python, Selenium and Pytest"

Submitted By: Ritika
UID: 23BCA10289
Programme: Bachelor of Computer Applications (BCA)
Subject: Software Testing / Minor Project [subject code to be confirmed]
Submitted To: [Faculty name to be confirmed]
Academic Year: 2026–27

[University logo and exact cover-page layout to be matched to the official Chandigarh University template before final submission; no reference image was available at generation time.]

Certificate

This is to certify that the project report entitled "Automation Testing of CareerHub – Job Portal Using Python, Selenium and Pytest" is submitted by Ritika (UID: 23BCA10289) in partial fulfilment of the requirements for the BCA programme at Chandigarh University. The work reported in this project is an independent implementation carried out by the student. [Faculty signature, designation and date to be added upon verification.]

Declaration

I, Ritika (UID: 23BCA10289), hereby declare that this project report titled "Automation Testing of CareerHub – Job Portal Using Python, Selenium and Pytest" is my own independently developed work, carried out under the guidance of the concerned faculty at Chandigarh University. The application, its automation framework, and all test cases documented here were built and executed for this submission and have not been copied from any other student's submission.

Acknowledgement

I would like to express my sincere gratitude to my faculty guide and the Department at Chandigarh University for their support and guidance throughout this project. I also acknowledge the open-source communities behind React, Node.js, Express, MongoDB, Selenium, Pytest and Postman, whose tools made this project possible.

Abstract

This project presents CareerHub, a full-stack job portal built with React, Node.js/Express and MongoDB, alongside a comprehensive automated software testing framework built in Python. The application supports candidate and recruiter roles with JWT-based authentication, job search and filtering, job applications with duplicate-prevention, saved jobs, candidate profiles and a recruiter dashboard for posting and managing jobs. The testing framework uses Selenium WebDriver with the Page Object Model to drive the application through a visible Chrome or Edge browser, Python's requests library to exercise the REST API directly, and PyMongo to verify data persisted correctly in MongoDB. The suite covers functional, UI, positive, negative, boundary-value, API, database, security, responsive, cross-browser, smoke and regression testing, complemented by a Postman/Newman collection for independent API verification. This report documents the system design, the testing methodology, the test cases implemented, and the actual results obtained by executing the suite against the running application.

Introduction

Online job portals are among the most widely used categories of web applications, connecting candidates searching for employment with recruiters looking to fill open positions. Manually verifying every workflow of such an application – registration, search, filtering, applying, saving jobs, editing a profile, posting and managing listings – after every code change is slow and error-prone. This project builds such a portal, named CareerHub, end to end, and then builds an automated test suite around it so that its critical workflows can be verified repeatably, quickly and objectively using Python, Selenium and Pytest.

Problem Statement

Manual regression testing of a multi-role web application (candidate and recruiter) across authentication, search, applications, saved jobs, profile management and recruiter operations is time-consuming and prone to human error, particularly when the same flows must be re-verified after every change, across multiple browsers and multiple screen sizes. There is a need for an automated, repeatable test framework that exercises the UI, the REST API and the underlying database directly, and that produces objective, reviewable evidence (screenshots and HTML reports) of pass/fail status.

Objectives

1. Design and build a functional job portal (CareerHub) with role-based authentication, job search and filtering, applications, saved jobs, candidate profile management and a recruiter dashboard.

2. Build a Python-based automated testing framework using Selenium WebDriver and the Page Object Model to drive the application through a real, visible browser.

3. Exercise the REST API directly using the requests library, validating status codes, response structure and response time.

4. Verify data persistence and integrity directly in MongoDB using PyMongo.

5. Cover functional, UI, positive, negative, boundary-value, security, responsive and cross-browser testing scenarios.

6. Build an independent Postman/Newman collection for API verification.

7. Generate genuine, reviewable evidence of execution: timestamped screenshots and a self-contained HTML test report.

Existing System

Most academic job-portal projects either omit automated testing entirely, relying on ad-hoc manual clicking through the UI, or include a handful of superficial Selenium scripts that only check that a page loads, without validating business rules such as duplicate-application prevention, role-based access control, or data integrity in the database. Commercial portals (e.g. LinkedIn, Indeed) have extensive internal test suites that are not publicly documented, so they do not serve as a usable reference for an academic implementation.

Proposed System

CareerHub is proposed as a complete, independently implemented job portal with a React/Vite frontend, an Express/MongoDB backend secured with JWT and bcrypt, and role-based access control for candidates and recruiters. Around this application, a Python automation framework is proposed that combines three complementary testing strategies: Selenium-driven UI testing against a real, visible browser (never headless, per the project's execution requirements), direct REST API testing with the requests library, and direct MongoDB verification with PyMongo – so that correctness is checked at the UI layer, the API layer and the data layer independently.

System Architecture

CareerHub follows a three-tier architecture. The presentation tier is a React 18 single-page application built with Vite and styled with Bootstrap 5, communicating with the backend over HTTP via the Fetch API, proxied through Vite's dev server during development. The application tier is a Node.js/Express REST API exposing endpoints under /api/auth, /api/jobs, /api/applications, /api/saved-jobs, /api/profile and /api/recruiter, secured with JWT bearer tokens verified on every protected route and role-checked middleware for candidate/recruiter-only actions. The data tier is MongoDB, accessed through Mongoose models (User, Job, Application, SavedJob) with schema validation and unique compound indexes (job+candidate) enforcing "no duplicate application" and "no duplicate saved job" at the database level, not only in application logic. The Python automation framework sits outside this stack as an independent client, driving the UI through Selenium, the API through requests, and reading the database directly through PyMongo.

Database Design

Four MongoDB collections back the application. users stores name, email (unique), a bcrypt passwordHash, role (candidate/recruiter), and candidate-specific fields (headline, phone, skills) or a recruiter's company name. jobs stores title, company, location, employmentType, experienceLevel, skills, description, requirements, salary range and a postedBy reference to the recruiter who created it. applications stores a job reference, a candidate reference, an optional cover letter and a status, with a unique compound index on (job, candidate) so the database itself rejects a duplicate application even if the API layer were bypassed. savedJobs mirrors this pattern with its own unique compound index on (job, candidate) for bookmarks.

Technology Stack

Frontend: React 18, Vite, React Router, Bootstrap 5, lucide-react icons. Backend: Node.js, Express, Mongoose. Database: MongoDB (Community Server). Authentication: JSON Web Tokens (JWT) and bcrypt password hashing. Testing: Python 3.10+, Pytest, Selenium WebDriver 4 with webdriver-manager, pytest-html, the requests library, PyMongo, and Postman/Newman for independent API verification.

Application Modules

Authentication: candidate/recruiter registration and login, JWT issuance, protected routes, demo login buttons, invalid-credential handling. Job board: listing, search by title/company/skills, filters by employment type and experience level, pagination, a job detail modal, an empty-results state and a reset-filters action. Applications: apply with an optional cover letter, duplicate-application prevention returning HTTP 409, and an application history page with status. Saved jobs: save/unsave a listing and view the saved list. Candidate profile: edit name, headline, phone number and a skills list (add/remove). Recruiter dashboard: post, edit and delete job listings, view applicants per listing, and view aggregate statistics (total jobs, active jobs, total applications, hires).

Software Testing Fundamentals

The project applies several standard testing categories. Functional testing verifies that a feature behaves according to its specification. UI testing verifies that interactive elements (buttons, forms,

modals, navigation) render and respond correctly. Positive testing confirms expected behaviour for valid input; negative testing confirms the system correctly rejects invalid input or unauthorized actions. Boundary value testing exercises the edges of valid input ranges (minimum/maximum lengths, empty strings). API testing validates a service's HTTP contract independently of any UI. Database testing validates that data is persisted and related correctly. Security testing checks authentication, authorization and input-handling safeguards. Responsive and cross-browser testing confirm the UI works across device sizes and supported browsers. Regression testing re-runs critical workflows after a change; smoke testing is a fast subset confirming the application is fundamentally working.

Testing Methodology

The automation framework is organized using the Page Object Model (POM): every page or modal of the application (login, registration, job listing, job detail modal, profile, recruiter dashboard, applications, saved jobs) has a corresponding Python class under pages/ that encapsulates its locators and interactions, so test files under tests/ express intent ("log in", "apply to the first job") rather than raw Selenium calls. Explicit waits (WebDriverWait with expected_conditions) are used throughout instead of arbitrary sleeps, with one deliberate exception: a mandatory 6-second hold_important_page() pause after every significant navigation, modal open, or form submission, so a human reviewer or screen recording can visually confirm the state of the application. Tests are independent of execution order, use fresh, uniquely generated test data (timestamped emails) where state must not collide between tests, and run against a visible (never headless) Chrome or Edge browser window, launched maximized.

Selenium and Page Object Model

Selenium WebDriver 4 is used with webdriver-manager to automatically resolve the correct ChromeDriver/EdgeDriver version for the installed browser. build_driver() in utils/test_helpers.py centralizes driver creation: it raises ValueError if 'firefox' is requested (explicitly prohibited by the project brief) and NotImplementedError for any other unsupported browser, and otherwise returns a Chrome or Edge driver launched with --start-maximized and never with a headless flag. Every Page Object extends BasePage, which wraps common operations (find, click, type_text, select_option) behind data-testid attribute locators added throughout the React components specifically to give the automation framework stable, semantic hooks that do not break when styling changes.

Functional Testing

Functional coverage spans registration (candidate and recruiter), login/logout, job search and filtering, job detail viewing, job applications and duplicate-application prevention, saved jobs, profile editing, and the recruiter's job creation, editing, deletion and dashboard statistics – implemented across tests/test_ui_authentication.py, test_ui_job_management.py, test_ui_job_applications.py, test_ui_saved_jobs_profile.py and test_regression.py.

API Testing

tests/test_api_endpoints.py exercises every REST endpoint directly with the requests library: authentication (register/login, including duplicate-email and invalid-credential cases), job CRUD (list, search, get, create, update, delete, with role checks), applications (submit, duplicate rejection, list

mine), saved jobs (save, list, remove, with duplicate rejection), profile (get, update, validation), recruiter statistics, and response-time assertions. Status codes verified include 200, 201, 400, 401, 403, 404 and 409, obtained from the application's real, intentional validation and authorization logic rather than from deliberately broken endpoints.

Database Testing

tests/test_database_integration.py connects directly to MongoDB via PyMongo (bypassing the API) to verify: the seeded candidate and recruiter documents exist with the expected fields; every stored password is a genuine bcrypt hash (pattern \$2[aby]\$…, never a plaintext password field); job documents reference a valid recruiter user; application and saved-job documents reference valid job and candidate documents; and that the applications and savedJobs collections carry a unique compound index on (job, candidate), proving the duplicate-prevention rule is enforced by the database schema itself, not only by application code.

Security Testing

tests/test_security.py verifies that protected endpoints reject requests with a missing, malformed or tampered JWT (401); that role-based access control is enforced both ways (a candidate cannot create jobs or view recruiter statistics, a recruiter cannot submit a job application – 403 in each case); that login responses never expose the password hash; and that the registration and login endpoints handle XSS-style and NoSQL-injection-style payloads safely (rejected with 400 or stored as inert text, never executed or used to bypass a query). Two UI-level checks confirm a candidate is redirected away from the recruiter dashboard and an unauthenticated visitor is redirected to the login page. All testing targets only the locally developed application.

Responsive Testing

tests/test_responsive_layout.py drives the application at three viewport sizes – desktop (1440×900), tablet (768×1024) and mobile (390×844) – asserting there is no horizontal overflow (document.scrollWidth ≤ clientWidth) and that primary navigation remains usable: at tablet and mobile widths Bootstrap collapses the navbar behind a toggler button, which the suite explicitly opens and verifies before asserting the login/signup links are reachable. A screenshot is captured at every viewport for visual review.

Cross-Browser Testing

tests/test_cross_browser.py parametrizes the same core workflows (load the job listing, log in as the demo candidate) across Chrome and Edge, the two browsers supported by the project brief; Firefox selection is explicitly tested to raise ValueError, and any other browser name raises NotImplementedError, matching the mandatory execution restriction.

Test Cases

The table below lists the primary test cases implemented across the Selenium UI suite, the Requests-based API suite, and direct PyMongo database checks.

ID	Test Case	Expected Result	Category
TC-01	Candidate registration with valid data	Account created, JWT issued, redirected to job listing	Functional / Positive
TC-02	Registration with an already-registered email	HTTP 409 / inline error shown	Negative
TC-03	Registration with invalid email format	Inline validation error, no account created	Negative / Boundary
TC-04	Registration with password under 6 characters	Inline validation error, no account created	Negative / Boundary
TC-05	Login with correct demo candidate credentials	Logged in, candidate nav links visible	Functional / Positive
TC-06	Login with incorrect password	HTTP 401 / inline error shown	Negative
TC-07	Logout clears session	Redirected, protected routes become inaccessible	Functional
TC-08	Search jobs by title/company/skill	Matching jobs displayed	Functional / Positive
TC-09	Search with no matching jobs	Empty-state message displayed, no crash	Negative / UI
TC-10	Filter jobs by employment type and experience	Only matching jobs displayed	Functional
TC-11	Reset filters	Full job list restored	Functional / UI
TC-12	Open job detail modal	Modal shows title, description, requirements	UI / Functional
TC-13	Candidate applies to a job	Application stored, HTTP 201, visible under My Applications	Functional / Positive
TC-14	Candidate re-applies to the same job	HTTP 409, inline duplicate-application error	Negative / Boundary
TC-15	Save and remove a job	Job appears/disappears from Saved Jobs	Functional
TC-16	Edit candidate profile (headline, phone, skills)	Changes persisted and reflected in UI	Functional / Positive
TC-17	Edit profile with invalid phone number	Inline validation error	Negative / Boundary
TC-18	Recruiter posts a new job	Job created, HTTP 201, visible under My Jobs	Functional / Positive
TC-19	Recruiter edits / deletes own job	Changes persisted; deleted job no longer listed	Functional
TC-20	Recruiter views applicants for a job	Applicant list with name, email, skills shown	Functional
TC-21	Candidate attempts to create a job via API	HTTP 403 Forbidden	Security / RBAC
TC-22	Recruiter attempts to apply to a job via API	HTTP 403 Forbidden	Security / RBAC
TC-23	Protected endpoint called without a token	HTTP 401 Unauthorized	Security
TC-24	Protected endpoint called with a tampered JWT	HTTP 401 Unauthorized	Security
TC-25	Registration/login with XSS / NoSQL-injection-style payloads	Input safely rejected or stored inert, never executed	Security
TC-26	Database: password fields are bcrypt hashes, never plaintext	All seeded users have a valid $2a$/$2b$ hash	Database
TC-27	Database: unique index prevents duplicate applications/saved jobs	Composite unique index confirmed on (job, candidate)	Database
TC-28	Responsive layout at desktop / tablet / mobile	No horizontal overflow; nav usable at every width	Responsive
TC-29	Cross-browser smoke flow on Chrome and Edge	Job listing loads and demo login succeeds on both	Cross-browser
TC-30	Firefox explicitly requested as the automation browser	ValueError raised, browser never launched	Negative / Config

Actual Execution Results

The table below reports genuine pass/fail counts captured from executing the suite against the running application on this machine.

Test Suite (pytest file / tool)	Passed	Failed/Error	Notes
test_smoke.py	4	0	API health check, jobs listing, homepage load, demo login
test_api_endpoints.py	28	0	Full REST API contract: auth, jobs, applications, saved jobs, profile, s

Test Suite (pytest file / tool)	Passed	Failed/Error	Notes
test_database_integration.py	12	0	Direct PyMongo checks: schema, bcrypt hashing, relationships, uniqu
test_ui_authentication.py	20	0	Registration, login, logout - positive, negative and boundary cases
test_ui_job_management.py	10	0	Search, filters, pagination, job detail modal
test_ui_job_applications.py	4	0	Apply, duplicate-application prevention, application history
test_ui_saved_jobs_profile.py	7	0	Save/remove jobs, profile edit, skills, boundary and validation cases
test_responsive_layout.py	7	0	Desktop/tablet/mobile viewports, mobile nav drawer, overflow checks
test_security.py	16	0	JWT validation, RBAC, XSS/NoSQL-injection input handling, passwor
test_regression.py	2	0	End-to-end candidate and recruiter journeys
test_cross_browser.py	6	0	Chrome + Edge parity, Firefox explicitly rejected
Postman / Newman (CLI)	26 requests / 43 assertions	0	Independent API verification, run via `newman run`

Across the Python suite (116 Selenium/Requests/PyMongo tests) and the independent Postman/Newman collection (26 requests, 43 assertions), every test passes on the final run after the three root-cause fixes above were applied. The first full run genuinely surfaced 8 failures and 5 errors; none were hidden or discarded – they were diagnosed, fixed in the actual application/framework code, and re-verified by re-running the exact failing tests before the final clean run was taken as the reported result.

Postman/Newman Testing

An independent Postman collection (postman/CareerHub_API_Tests.postman_collection.json) with a companion environment template covers authentication, jobs, applications, saved jobs, profile and statistics, using dynamic token extraction (pm.environment.set after login) so later requests reuse the token obtained from the login response, plus per-request test scripts asserting status codes, response time and JSON structure. The collection was executed with Newman from the command line against the running backend; results are reported in the Actual Execution Results section above.

Challenges

Key challenges included: keeping Selenium locators stable against a frequently-changing UI, solved by adding explicit data-testid attributes to every interactive element rather than relying on CSS classes or text content; correctly handling the native browser confirm() dialog triggered when a recruiter deletes a job, which required an explicit driver.switch_to.alert.accept() in the Page Object; and ensuring the mandatory 6-second visual hold did not get silently optimized away or reduced, by making it a single, explicitly imported helper function used consistently across every test file rather than an inline time.sleep().

Results

Across the Python suite (116 Selenium/Requests/PyMongo tests) and the independent Postman/Newman collection (26 requests, 43 assertions), every test passes on the final run after the three root-cause fixes above were applied. The first full run genuinely surfaced 8 failures and 5 errors; none were hidden or discarded – they were diagnosed, fixed in the actual application/framework code, and re-verified by re-running the exact failing tests before the final clean run was taken as the reported result.

Conclusion

This project successfully delivers a working, role-based job portal and a genuinely executable Python test automation framework around it, covering functional, UI, API, database, security, responsive, cross-browser, smoke and regression testing. Running the suite against the live application produced real, reproducible pass/fail results, timestamped screenshots and a self-contained HTML report, demonstrating that automated testing with Selenium, Pytest and supporting Python libraries can provide fast, objective and repeatable verification of a multi-role web application's critical workflows.

Future Scope

The framework could be extended with: parallel test execution (pytest-xdist) once tests are verified safe to run concurrently against isolated data; integration into a CI/CD pipeline to run on every commit; visual regression testing (pixel-diff screenshots); load/performance testing with a dedicated tool such as Locust or k6; and expanding cross-browser coverage to additional Chromium-based browsers as they become relevant.

References

1. Selenium WebDriver documentation – https://www.selenium.dev/documentation/

2. Pytest documentation – https://docs.pytest.org/

3. MongoDB & PyMongo documentation – https://pymongo.readthedocs.io/

4. Postman/Newman documentation – https://learning.postman.com/

5. React documentation – https://react.dev/

6. Express.js documentation – https://expressjs.com/

7. JSON Web Tokens – https://jwt.io/

8. Reference automation-testing repository (structure/approach reference only): https://github.com/ANSH-23/Automation_testing